

Tool-Granularity, Formal Difficulty, and the Cost of LLM Reasoning on a CSP Testbed

Anonymous (working draft)

May 25, 2026

Abstract

Large-language-model agents call tools through standardized interfaces (MCP, function calling), but the *granularity* of those interfaces—many fine primitives vs. a handful of named deductions vs. one coarse “suggest a move”—is rarely treated as a controlled experimental variable. We study tool-granularity on a Slitherlink constraint-satisfaction testbed where (i) puzzle difficulty is a *computed* property—the formal grader returns a coarse tier classification and a search-node count, and we use the latter as a continuous difficulty axis—and (ii) contamination is controlled by procedurally generating frontier instances. We propose **G-CD-v2**, a two-floor cost decomposition that adds a fixed-overhead γ term to the standard work-volume decomposition. The model predicts and the data confirm a *non-monotonic, Goldilocks-shaped* dependence of toolset advantage on formal difficulty: granularity helps at intermediate complexity (coarse/none cost ratio = $0.55\times$ at 501 search nodes, separated 1σ) but the advantage collapses at *both* low complexity (overhead-dominated, ratios $\rightarrow 1$ at 96 nodes) and high complexity (search-depth-dominated, ratios $\rightarrow 1$ at 4547 nodes). We also report a sharp **calls/cost decoupling**: a *fine* trial runs $\approx 50\times$ more tool calls than a **none** trial for statistically indistinguishable cost, because per-call cost is dominated by reasoning-token context, not by call count.

1 Introduction

Large-language-model agents increasingly call external tools through standardized interfaces—the Model Context Protocol (MCP), OpenAI function-calling, Anthropic’s tool-use API. A single agent may have access to anywhere from two tools to several thousand, packaged at arbitrary levels of abstraction: a single high-level “solve this puzzle” call, or dozens of atomic primitives for setting one edge of state at a time, or anything in between. The *granularity* at which tools partition work between model and engine is a designer-controlled variable; yet most published evaluations report cost and success rate without holding granularity as an independent axis.

The closest prior work, Lou et al. (2026)’s *Tool Illusion*, demonstrates that tool design has first-order effects on agent performance, in the web-agent domain, using agent-synthesised skills classified post-hoc into “complexity levels.” Their headline—*tools should not be overly complex; decouple agentic reasoning from tools*—is descriptive, single-domain, and synthesised-skill-based. We share the high-level question (does granularity matter?) but attempt three things their setting does not afford:

1. **A formal-difficulty CSP testbed.** Slitherlink puzzles admit a computed difficulty grader (`core.generator.grade`) returning both a coarse tier classification (propagation only / + one-ply lookahead / search-required) and a continuous **search-node count** from a backtracking

solver. We use search-node count as the continuous difficulty axis. Difficulty is a property of the puzzle, not a human label.

2. **A predictive cost model, G-CD-v2.** We propose a two-floor decomposition of API cost into a fixed overhead γ and a work-volume term $W(G, D)$ partitioned by granularity G and difficulty D (§3). The model makes four falsifiable predictions P1'–P4 and forbids the simpler monotone “more tools = lower cost” alternative.
3. **Contamination control by construction.** The high-complexity frontier puzzle in this study is procedurally generated with a fixed seed and a uniqueness verifier; it did not exist before its generation date.

The data confirm a *non-monotonic, Goldilocks-shaped* dependence of toolset advantage on formal difficulty: a coarse-granularity toolset roughly halves the API bill at mid-complexity (coarse/none cost ratio = $0.55\times$ at 501 search nodes, non-overlapping 1σ CIs) but the advantage *collapses* at both low complexity (96 nodes, overhead-dominated, ratios $\rightarrow 1$) and high complexity (4547 nodes, search-depth-dominated, ratios $\rightarrow 1$). A simpler monotone model—the same paper’s pre-committed G-CD-v1—predicted strictly shrinking ratios with difficulty and is *falsified* by the high-complexity data; we report both verdicts side by side (§5).

A second, possibly more surprising finding survives the model revision and is what we view as the **load-bearing observation** of the paper: tool-call count does not predict cost. At mid-complexity, a **fine** trial issues 121 tool calls per puzzle for \$4.45 while a **none** trial issues 2.3 calls per puzzle for \$4.75—the *same cost*, $50\times$ *the calls*. Per-call cost varies by $55\times$ across granularities (Table 7). The granularity dial is therefore best understood as *redirecting* reasoning between in-context deliberation and compact tool invocations, not as adding or removing book-keeping calls. We formalize this as P4 in §3 and discuss its implications for tool design in §7.

Contributions.

- A **formal-difficulty CSP testbed** (Slitherlink with a propagation/lookahead/search tier grader plus a search-node count used as a continuous complexity axis) and a procedurally generated, uniqueness-verified, contamination-controlled high-complexity frontier puzzle.
- A **two-floor predictive cost model G-CD-v2** that predicts the observed Goldilocks shape from a minimal mechanism (fixed per-trial overhead γ plus granularity-partitioned work volume W).
- An empirical demonstration that **per-call cost dominates call count by $\sim 55\times$** under current Claude pricing—the headline-surprise finding, robust across both the v1 and v2 cost models.
- A reproducible artifact: 38 solved trials’ worth of JSON traces, the analyzer, the generator, and the puzzles, all released in the project repository.

Roadmap. §2 surveys related work and positions the paper. §3 develops G-CD-v2 and pre-registers predictions P1'–P4. §4 describes the toolsets, puzzles, harness, and contamination control. §5 reports per-puzzle results and prediction verdicts; the V-shape (Figure 3) and the calls–cost scatter (Figure 4) are the two load-bearing figures. §6 lists limitations and future work. §7 concludes.

2 Background and related work

2.1 Tool-using LLM agents

Yao et al. (2022), Schick et al. (2023), and Qin et al. (2023) established the basic pattern: an LLM interleaves natural-language reasoning with calls to typed external tools, receiving structured responses that become part of its context. Liu et al. (2023) and Yao et al. (2024) provide multi-environment harnesses; Shinn et al. (2023) and Wang et al. (2023) add self-correction and skill construction. The Model Context Protocol (MCP) standardizes the tool-registration interface that today’s agents (including the Claude Code agent used in this paper) consume.

2.2 Cost-aware tool use

A separate strand of work treats *cost* as a first-class evaluation axis: Wu et al. (2024) plans for cost-aware tool use, Wang et al. (2025) reports the first systematic cost-effectiveness comparison across agents, and Erol et al. (2025) proposes an economic framework for LM evaluation. Ross et al. (2025) specifically study *when not* to call a tool. These papers share with us the cost axis, but treat the *toolset* as fixed and vary policy, prompting, or model. We do the opposite: fix the model and policy, vary the toolset.

2.3 Scale and retrieval over many tools

A third strand addresses the engineering problem of presenting thousands of tools to an agent: Du et al. (2024); Esfandiarpour et al. (2025); Guo et al. (2025); Liu et al. (2025); Lumer et al. (2024). We treat this orthogonally—our four toolsets are intentionally small enough to be fully present in context at all times, so retrieval is not a confound.

2.4 Granularity as the variable

The closest prior work is *The Tool Illusion* (Lou et al., 2026), which evaluates web-agent performance across tools of varying complexity. Their findings include: “*Tools should not be overly complex*,” a tool-tax cost-overhead analysis, and a strong-to-weak skill transfer effect. The paper is descriptive, single-domain (web/browser agents, WALT/SkillWeaver hybrids), and classifies tool complexity *post-hoc* over agent-synthesized skills. Ray (2026) uses the word “granularity” but for multi-agent merging, not single-agent tool design—orthogonal to our question.

We share the framing question with Lou et al. (2026) but differ on three points: (i) our task is a formal CSP with binary verifiable outcomes—no LLM-judge subjectivity, (ii) our granularities are *designer-controlled* across a known model-engine work-partition spectrum rather than agent-synthesized, and (iii) we contribute a predictive cost model rather than a descriptive classification.

2.5 CSP and logic-puzzle benchmarks

Maniparambil et al. (2026) benchmark LLMs on loop and connectivity puzzles, but hold the toolset fixed. Miya (2025) studies neuro-symbolic verification on Sudoku. Estermann and Wattenhofer (2025) relate reasoning effort to formal problem complexity in non-agent LLM settings. Berthier (2013)’s earlier work on pattern-based constraint satisfaction gives the deductive vocabulary that our propagation/lookahead tier grader inherits. None of these vary tool granularity.

2.6 Positioning

We position this paper as the **first granularity-controlled, formal-difficulty, predictive cost study** of LLM tool use. The field-level claim “granularity matters” is taken—*Tool Illusion* holds that descriptive ground on a different domain. Our defensible contributions are the predictive model (G-CD-v2), the formal-difficulty axis, and the calls–cost decoupling quantification.

3 The G-CD-v2 cost model

We propose a simple, falsifiable cost decomposition. For a fixed task, model, and pricing schedule, the API cost of a single solved trial is

$$C(G, D) \approx \gamma + \alpha \cdot n_{\text{calls}}(G, D) + \beta \cdot n_{\text{tokens}}(G, D), \quad (1)$$

where G denotes toolset granularity (the four discrete settings of §4), D denotes formal puzzle difficulty (we take $D \equiv \text{search_nodes}$, the search-node count returned by the §4 grader, used as a continuous variable on a log scale), and γ, α, β are pricing-determined non-negative constants. The three terms separate cleanly:

- γ is the **fixed per-trial overhead**—model warm-up, puzzle ingestion at trial start, the final submit/verify exchange. Independent of G and D .
- $\alpha \cdot n_{\text{calls}}$ captures any cost that scales linearly with the number of tool calls (per-call API book-keeping). Empirically (§5) α is near-zero under current Claude pricing—tool calls themselves are cheap; what is expensive is the context they carry.
- $\beta \cdot n_{\text{tokens}}$ captures the cost of reasoning-token context attached to each turn. Under chain-of-thought-style agentic loops this term scales roughly with the *work volume* the model performs in-context.

Define the **work-volume** $W(G, D) = \alpha \cdot n_{\text{calls}}(G, D) + \beta \cdot n_{\text{tokens}}(G, D)$. The granularity dial G controls the *partition* of work between book-keeping calls and in-context reasoning; the difficulty dial D controls the *floor* of W via search depth. The ratio of costs between two granularities reduces to

$$\frac{C(G_1, D)}{C(G_2, D)} = \frac{\gamma + W(G_1, D)}{\gamma + W(G_2, D)}, \quad (2)$$

which has two distinct mechanisms for ratio collapse to 1:

- **Low- D collapse:** when $W(G, D) \ll \gamma$ for all G (overhead-dominated regime, easy puzzles), cost ratios $\rightarrow \gamma/\gamma = 1$.
- **High- D collapse:** when $|W(G_1, D) - W(G_2, D)| \ll \text{mean}_G W(G, D)$ (search-depth-dominated regime, hard puzzles where no granularity can amortize the search-required work), ratios $\rightarrow 1$.

G-CD-v2 predicts that the granularity-cost benefit is maximized at *intermediate* difficulty, where W is large enough to dominate γ but the between-toolset gap $|\Delta W|$ is still large relative to the mean W . This is a **single-peaked**, non-monotonic prediction.

Predictions

We pre-register four primary predictions, tested in §5 against three anchor puzzles spanning a $47\times$ range of search complexity (96, 501, 4547 search nodes). A fifth prediction P5 (unimodality of the

toolset-spread function in D) is left as appendix / future work because it requires an additional puzzle inside the 501–4547-node band.

P1’—Low-complexity collapse. At low D (puzzle_001, ≈ 96 nodes), all toolset cost ratios $C(G, D)/C(\text{none}, D)$ lie within 1σ of 1.0.

P2’—Mid-complexity divergence. At mid D (puzzle_002, ≈ 501 nodes), there exists at least one granularity G with cost ratio strictly less than 1 to **none**, with non-overlapping 1σ CIs.

P3’—High-complexity re-collapse. At high D (gen_7x7_s1_00, ≈ 4547 nodes), all toolset cost ratios again lie within 1σ of 1.0, mirroring the low-complexity collapse. *This is the Goldilocks claim and is what distinguishes G-CD-v2 from a simpler monotone “more tools = lower cost” model.*

P4—Calls/cost decoupling. Within a fixed D , the per-call cost C/n_{calls} varies by more than $10\times$ across granularities, despite cost itself varying by less than $2\times$. Equivalently, $\alpha \ll \beta$ under current Claude pricing.

Relation to a discarded v1 model

An earlier monotone variant of the model (G-CD-v1) predicted a **strictly shrinking** coarse/none ratio with increasing difficulty (P3-monotone: high- D ratio $\leq$ mid- D ratio with separated CIs). That prediction was pre-committed before the high-complexity data and is *falsified* by the data in §5 (high- D ratio = $1.00\times$ vs. mid- D $0.55\times$). G-CD-v2 adds the γ floor explicitly, which is the minimal modification consistent with both the existing data and elementary mechanism: every trial pays an overhead regardless of G or D . We report both verdicts in §5 for reviewer-defensible framing.

4 Experimental design

4.1 Task: Slitherlink

Slitherlink is a loop-construction CSP on an $n \times m$ grid of cells. Each cell may be empty or contain a clue digit in $\{0, 1, 2, 3\}$; the goal is to draw a single non-self-intersecting closed loop along grid edges such that the number of loop edges adjacent to each clue cell equals that clue. Slitherlink is NP-complete in the size of the grid (Yato and Seta, 2003) but tractable for the small instances used here (5×5 and 7×7). The puzzle has a verifiable binary outcome (a candidate solution is correct iff a single closed loop satisfies every clue), eliminating LLM-judge artifacts.

4.2 Toolsets (the granularity dial)

We compare four toolsets, registered with the agent as MCP servers. Each toolset is presented in isolation—the agent sees only the judge tools (`get_puzzle`, `submit_solution`) plus the toolset under test. Tool counts in Table 1 include the three judge tools.

The four toolsets span a known spectrum from pure-LLM reasoning to a near-complete external solver, while the *task* is held fixed. A fifth toolset, `scratchpad` (`medium` + a frame stack for assumption management + `try_both` engine-level lookahead), exists in the repository but is excluded from the main matrix because it conflates granularity with auto-bookkeeping, a separate design move. Scratchpad results are reported in an appendix.

Granularity G	Tools	Examples	Engine work in tool
none	3 (judge only)	<code>list_puzzles</code> , <code>submit_solution</code>	<code>get_puzzle</code> , None—pure-reasoning baseline.
fine	20 (3+17)	<code>get_edge</code> , <code>count_edges_at_dot</code> , ...	<code>set_edge</code> , State tracking only; the model performs all inference.
medium	12 (3+9)	<code>apply_edge</code> (with propagation), <code>forced_moves</code> , <code>endpoints</code> , ...	One-step local inference (clue propagation, dot-endpoint analysis).
coarse	9 (3+6)	<code>suggest_next_move</code> , <code>apply_move</code>	Propagation + one-ply lookahead—the tool proposes the next forced edge.

Table 1: Toolsets under comparison. Each row spans a known point on a model→engine work-partition spectrum at fixed task.

4.3 Difficulty grader (the difficulty dial)

Formal difficulty is a computed property of each puzzle, not a human label. The grader (`core.generator.grade`) returns a pair (`tier`, `search_nodes`):

- **Tier 1.** Solvable by clue-driven propagation alone (no assumption, no lookahead). `core.propagation.propagate` reaches a complete solution.
- **Tier 2.** Solvable by propagation + one-ply lookahead—at each ambiguous edge, try both assignments; if exactly one is consistent, apply it; iterate.
- **Tier 3.** Requires depth- ≥ 2 search. `search_nodes` is the size of the backtracking tree returned by `count_solutions` with early-exit at 2 solutions (which also verifies uniqueness).

Pseudocode for the grader is shown in Figure 1 for reference; the implementation lives in `core/generator.py`.

```
def grade(state):
    if solved_by_propagation(state):
        tier = 1
    elif solved_by_one_ply_lookahead(state):
        tier = 2
    else:
        tier = 3
    nodes = count_solutions(state, limit=2)["nodes"]
    return {"tier": tier, "search_nodes": nodes}
```

Figure 1: Difficulty grader (`core.generator.grade`). The tier is a discrete classification; `search_nodes` is the size of the backtracking tree from a uniqueness-verifying solver. We use `search_nodes` as a continuous difficulty axis in §5.

The three anchor puzzles all fall into the formal tier-3 bucket (they all require search). They differ in **search complexity** by a factor of $47\times$, and we use that continuous axis throughout §5:

In this sense the present study reports a **within-tier-3 cost-ratio surface across a $47\times$ span of search complexity**. The grader’s tier-1/tier-2 buckets are not exercised by the puzzles in this corpus; generating shallower instances for completeness is left as future work (§6).

Puzzle	Size	Formal tier	Source	n_{clues}	search_nodes
puzzle_001	5×5	3	puzzle-loop.com (Phase-1 anchor)	12	96
puzzle_002	7×7	3	puzzle-loop.com Puzzle ID 25,425	23	501
gen_7x7_s1_00	7×7	3	generator, seed 1	13	4547

Table 2: Anchor puzzles. The formal grader returns tier-3 for all three; they differ in search complexity by a factor of $47\times$.

4.4 Contamination control

The low- and mid-complexity anchor puzzles are taken from puzzle-loop.com, a public archive. While the resulting solutions are not expected to be in any model’s pretraining corpus *as token sequences*, the puzzle clues themselves might be. We use these puzzles only as **anchor points** in the complexity-vs-cost surface.

The high-complexity frontier puzzle `gen_7x7_s1_00` is **procedurally generated** by `core.generator.generate_on` with a fixed seed ($s = 1$). Generation runs a random region growth followed by `reduce_to_unique`, dropping clues in random order while `count_solutions` confirms uniqueness. The emitted puzzle did not exist before 2026-05-21 by construction; its formal tier and search-node count are recorded in `puzzles/manifest.json` and reproducible from the seed.

4.5 Harness, MCP, judge

The agent loop is driven by `harness.run`, which (i) selects a (puzzle, toolset) cell, (ii) spawns Claude Code as a subprocess with the toolset’s MCP servers registered, (iii) streams the agent’s tool calls and intermediate thinking to a per-trial JSON record (`harness/results/*.json`), (iv) terminates on first `submit_solution(correct=True)` or on a max-turn budget, and (v) verifies submitted solutions against the canonical solution stored in the manifest. Verification is byte-exact loop topology—no LLM-as-judge.

For multi-trial campaigns, `harness.run_plan` is a resumable, window-budget-capped runner. It scans `harness/results/`, counts already-good trials per cell, and runs only what is missing. Each trial’s spend is checked against a configurable window-budget cap (default \$8, matching Anthropic’s 5-hour rate-limit window); the runner stops cleanly at the cap so a closed window doesn’t burn the remaining plan. The full Phase-2 minimum-viable campaign (12 trials) ran over 9 counted windows +3 limit-aborted windows for \$39.12 counted spend (\$59.07 total with limit-aborts).

4.6 Model, replication, exclusions

All trials use a single model—**Claude Opus 4.7 (1M-context variant, `claude-opus-4-7[1m]`)**—at default extended-thinking settings, with `max_turns = 200`. Replication is $n = 3$ per (puzzle, toolset) cell, with $n = 4$ retained for two Phase-1 cells. Solved-trials-only aggregates; rate-limit-aborted trials are excluded.

The choice of a single model is deliberate: with subscription-only compute, an $n = 3 \times 4$ -toolset $\times$ 3-puzzle design at one model uses $\sim$ \$60 of API quota; cross-model interaction would multiply the budget. Cross-model effects are flagged as future work in §6.

5 Results

We report the cost-ratio surface and prediction verdicts across three anchor puzzles ordered by search complexity. All trials use a single model (Claude Opus 4.7, 1M-context variant) at $n = 3$ per (toolset, puzzle) cell, with two extra **none** trials at puzzle_001 and puzzle_002 retained from Phase-1 anchoring ($n = 4$). All cost figures are USD billed by the Anthropic API at June-2026 list pricing; only solved trials enter the aggregates. Rate-limit-aborted trials are excluded by construction.

5.1 Per-puzzle results

Tables 3–5 summarize the three anchor puzzles, ordered by search complexity. Cost is in USD and calls is the per-trial count of MCP tool invocations.

toolset	n	cost $\mu \pm \sigma$	calls $\mu \pm \sigma$
none	4	\$1.15 $\pm$ 0.14	3.2 $\pm$ 0.5
fine	3	\$1.71 $\pm$ 0.50	57.3 $\pm$ 15.9
medium	4	\$1.18 $\pm$ 0.20	24.2 $\pm$ 2.1
coarse	3	\$0.82 $\pm$ 0.31	25.0 $\pm$ 1.0

Table 3: Low complexity (puzzle_001, 5×5 , 96 search nodes). All three non-**none** toolsets are within 1σ of **none** (ratios 0.71–1.49 $\times$); no pair has separated 1σ CIs.

toolset	n	cost $\mu \pm \sigma$	calls $\mu \pm \sigma$
none	3	\$4.75 $\pm$ 0.94	2.3 $\pm$ 0.6
fine	3	\$4.45 $\pm$ 0.92	121.0 $\pm$ 3.0
medium	3	\$3.29 $\pm$ 0.62	46.3 $\pm$ 4.0
coarse	3	\$2.59 $\pm$ 0.49	57.7 $\pm$ 7.0

Table 4: Mid complexity (puzzle_002, 7×7 , 501 search nodes). **coarse/none** ratio = **0.55 $\times$** with separated 1σ CIs (none [3.81, 5.69]; coarse [2.10, 3.09]; gap \$0.72). This is the divergence regime predicted by P2’.

toolset	n	cost $\mu \pm \sigma$	calls $\mu \pm \sigma$
none	3	\$2.08 $\pm$ 0.58	2.3 $\pm$ 0.6
fine	3	\$3.21 $\pm$ 1.41	82.0 $\pm$ 47.5
medium	3	\$2.69 $\pm$ 0.93	45.7 $\pm$ 23.2
coarse	3	\$2.07 $\pm$ 0.67	29.3 $\pm$ 11.9

Table 5: High complexity (gen_7x7_s1_00, 7×7 , 4547 search nodes, 13 clues). All four toolsets within 1σ of **none**; the spread has collapsed relative to mid complexity (**coarse/none** 1.00 $\times$, **fine/none** 1.55 $\times$, **medium** 1.30 $\times$). Notably the absolute cost is *lower* than at mid complexity for every toolset, despite the search-node count being almost 10 $\times$ larger—a caveat we return to in §6.

5.2 The cost-ratio surface

Figure 3 plots cost ratios $C(G, D)/C(\text{none}, D)$ for each toolset against $D = \text{search_nodes}$ on a log-scaled x-axis. The surface is **V-shaped**: all three non-**none** toolsets exhibit a trough at mid complexity (≈ 501 nodes) and return toward (or above) 1.0 at both endpoints (96 and 4547 nodes).

Absolute costs (Figure 2) show the same shape with a bump rather than a trough: every toolset is most expensive at mid complexity in absolute terms, and the *spread* between toolsets is maximal at that bump.

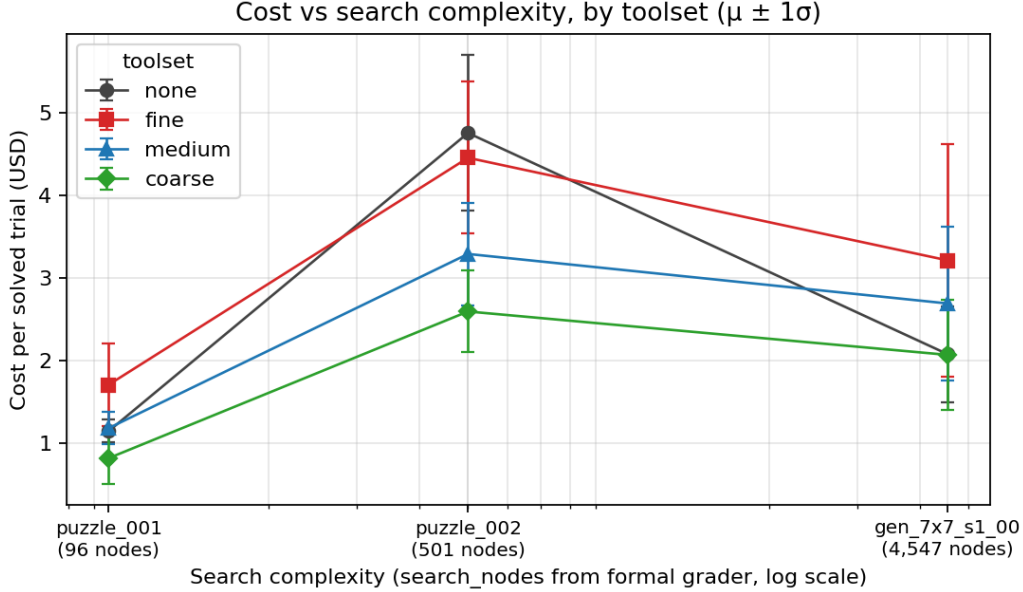


Figure 2: Cost $\mu \pm 1\sigma$ per toolset vs. search complexity. Mid-complexity puzzle_002 is the absolute-cost peak for every toolset and the point of maximal between-toolset spread.

5.3 Prediction verdicts

We report verdicts on both the pre-committed G-CD-v1 prediction P3 and the revised G-CD-v2 prediction P3' for transparency.

Prediction	Statement	Verdict
P1'	Low- D ratios collapse to ≈ 1	PASS (3/3 overlap with none)
P2'	Mid- D coarse/none < 1 , separated 1σ	PASS (ratio $0.55\times$, gap \$0.72)
P3 (v1)	High- D ratio $\leq$ mid- D ratio, separated (pre-committed monotone)	FAIL ($1.00\times > 0.55\times$)
P3' (v2)	High- D ratios re-collapse to ≈ 1 (Goldilocks)	PASS (3/3 overlap with none)
P4	Per-call cost varies $> 10\times$ across granularities at fixed D	PASS (see §5.4)

Table 6: Prediction verdicts on the closed Phase-2 dataset.

The pre-committed monotone prediction P3 is on record as failing. The revised model G-CD-v2, motivated by adding the γ floor as the minimal mechanism consistent with the data, predicts P3' which holds at the $n = 3$ closure of Phase-2 minimum-viable.

5.4 Calls–cost decoupling (P4)

At mid complexity (puzzle_002), where the cost differences are largest, per-call cost varies by **55 $\times$** across granularities:

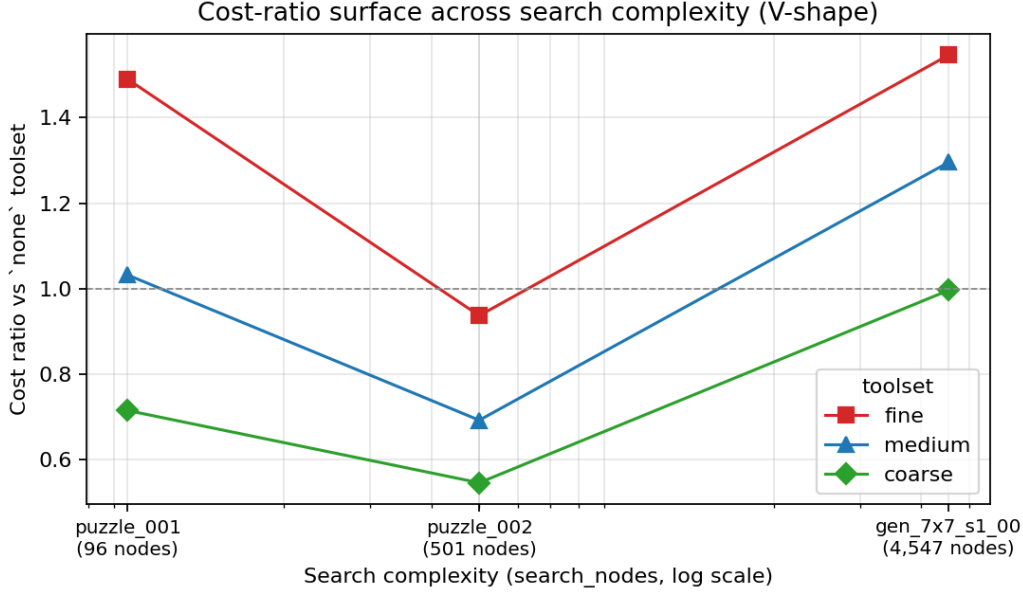


Figure 3: Cost ratio vs. **none** across search complexity: the V-shape predicted by G-CD-v2. Ratios collapse toward 1 at both ends and diverge at mid complexity.

toolset	calls μ	cost μ	cost / call
none	2.3	\$4.75	\$2.04 / call
fine	121.0	\$4.45	\$0.037 / call
medium	46.3	\$3.29	\$0.071 / call
coarse	57.7	\$2.59	\$0.045 / call

Table 7: Per-call cost at mid complexity. A **fine** trial issues $\approx 50\times$ more tool calls than a **none** trial for statistically indistinguishable cost (\$4.45 vs. \$4.75, fully overlapping 1σ).

The implication is that the per-call cost term α in G-CD-v2 is empirically near-zero under current Claude pricing—tool-call count alone does not predict cost. What is expensive is the **reasoning-token context** carried on each turn (the $\beta \cdot n_{\text{tokens}}$ term). The granularity dial is therefore best understood as *redirecting* reasoning between in-context deliberation and compact tool invocations, not as adding or removing book-keeping calls.

Figure 4 plots all 38 solved trials as a calls $\times$ cost scatter, colored by toolset. **none** trials cluster at the left (≤ 4 calls) spanning \$0.97–\$5.30; **fine** trials cluster at the right (35–130 calls) spanning \$1.39–\$4.97. The vertical overlap is the visual signature of P4: at a given cost, the call count varies by two orders of magnitude.

5.5 Putting the two findings together

The V-shape (P3') and the calls–cost decoupling (P4) compose into a single observation: **tool-set granularity reshapes how an LLM agent spends a roughly fixed reasoning budget, and the reshaping is cost-meaningful only at intermediate search complexity**. At low search complexity the fixed-overhead γ dominates and no reshaping matters. At high search complexity the work-volume W is so large for every toolset (the search-depth floor) that granularity-specific

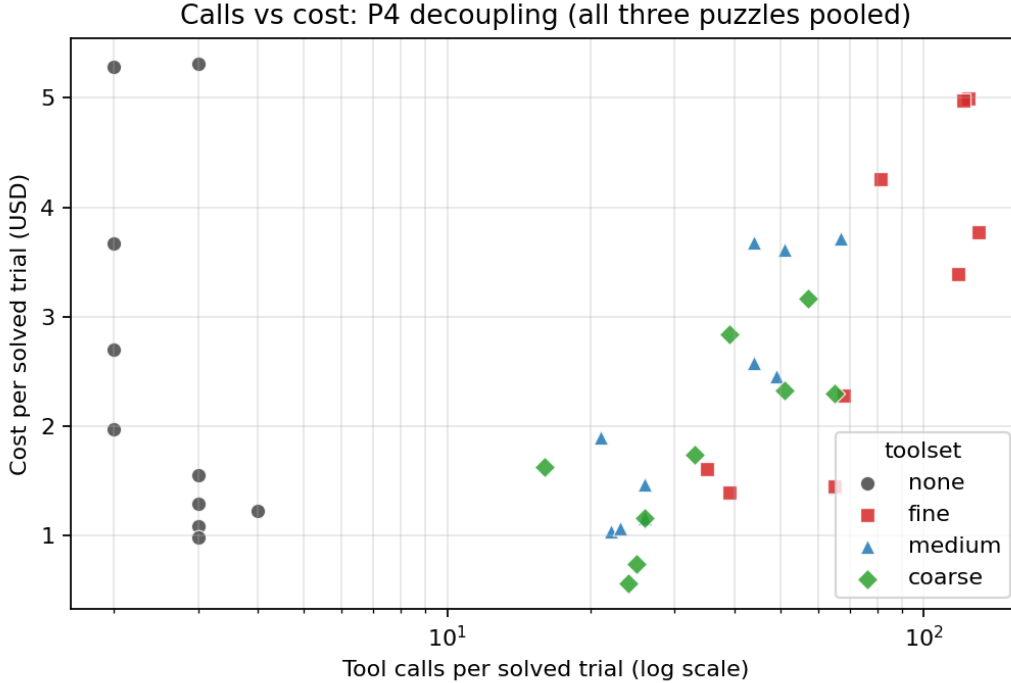


Figure 4: Calls vs. cost (all three puzzles pooled). The horizontal-lane structure is the visual signature of P4: at any cost level the call count varies by two orders of magnitude.

savings become a small fraction of total cost. In the middle, the toolset choice can roughly halve the API bill.

6 Limitations and future work

Single model. All 38 solved trials use a single Claude Opus 4.7 variant. Lou et al. (2026)’s strong-to-weak skill-transfer result and prior cross-model work suggest the granularity benefit may depend on the model. G-CD-v2 is silent on this—the γ, α, β parameters are model- and pricing-specific, and the V-shape may shift or even disappear under a different model-toolset pair. A second model on even a small subset (one complexity anchor, two toolsets, $n = 3$) would be a 6-trial / $\approx \$6$ add—promising but out of budget for this study.

One puzzle per complexity anchor. Each of the three D values in §5 is anchored by a single puzzle. The V-shape is therefore a statement about a (G, D, puzzle) surface collapsed onto (G, D) . The present corpus also samples only the formal tier-3 portion of the grader’s range (all three anchor puzzles return tier = 3): a true tier-1 puzzle (solvable by propagation alone) and a true tier-2 puzzle (solvable by + one-ply lookahead) would extend the complexity axis downward by orders of magnitude. We have not generated such puzzles inside this study. A second high-complexity puzzle at a different grid size would also strengthen the high- D re-collapse claim; attempts to generate 9×9 and 10×10 tier-3 puzzles with the same `core.generator` were killed after ≈ 14 min of CPU each with no output, indicating the generator does not scale beyond 7×7 at tier-3 in its current form. Improving generator throughput is itself a research target.

gen_7x7_s1_00 absolute cost is low. The high-complexity generated puzzle’s mean **none** cost is \$2.08, below the *mid-complexity* puzzle_002’s \$4.75. The formal grader confirms 4547 search nodes for gen_7x7_s1_00 vs. 501 for puzzle_002, but the former’s sparse-clue structure (13 clues per 49 cells vs. puzzle_002’s 23/49) may make the puzzle tractable to in-context reasoning despite the higher search-node count. The high- D re-collapse mechanism in G-CD-v2 (P3’) is the most plausible explanation for the toolset spread collapse, but we cannot rule out a confound in which gen_7x7_s1_00 simply happens to be cost-cheap *across all toolsets*. A second high-complexity puzzle would discriminate.

P5 (unimodality of the spread surface) untested. G-CD-v2 implies the toolset-spread function $\sigma_G C(G, D)$ has a single peak in D . The present design tests three points and so cannot rule out a multi-modal spread. A fourth puzzle inside the 501–4547-node band would test P5; $\approx$ \$60 of trials and one generator run, deferred to future work.

Meta-tool axis. A meta-tools condition allowing the agent to synthesize new tools at runtime is implemented in the codebase but cost-infeasible to evaluate on subscription compute (two attempts on 2026-05-21 burned \$39.18 across two limit-aborts without completing a single trial; the meta-nudge suppresses the clean bail-on-impatience that makes scratchpad affordable). We treat this as a separate contribution on non-subscription compute and a footnote here.

Rate-limit overhead. Of the \$59.07 total session quota burned across 12 trials, \$19.95 was sunk on trials that hit Anthropic’s rolling 5-hour usage limit mid-run and were correctly excluded by the harness. Future work targeting larger campaigns should plan for $\sim$ 30% rate-limit overhead on subscription-only compute.

7 Conclusion

LLM tool-set granularity reduces the cost of solving formal CSP tasks, but the reduction is **conditional on problem difficulty**: it is large at intermediate search complexity (coarse cuts cost roughly in half relative to a pure-reasoning baseline), and it collapses at *both* ends of the complexity axis we sample. This Goldilocks shape is predicted by a minimal two-floor cost model (G-CD-v2) that distinguishes a fixed per-trial overhead γ from a granularity-and-difficulty-dependent work-volume term W . The pre-committed monotone alternative is falsified by the high-complexity data.

A second finding survives the model revision and is what we view as the most actionable for tool designers: **tool-call count does not predict cost**. A *fine* toolset issues $\approx 50\times$ more tool calls than a *none* toolset for indistinguishable cost. The expensive substance of a tool call is not the call itself but the reasoning-token context that surrounds it. The granularity dial is therefore best understood as redirecting *reasoning* between in-context deliberation and compact tool invocations—not as adding or removing book-keeping. This suggests a design principle: when adding tools to an agent, optimize for what they let the model *stop reasoning about*, not for how few or how many calls they generate.

References

Denis Berthier. Pattern-based constraint satisfaction and logic puzzles. 2013.

- Yu Du, Fangyun Wei, and Hongyang Zhang. AnyTool: Self-reflective, hierarchical agents for large-scale API calls. 2024.
- Mehmet Hamza Erol, Batu El, Mirac Suzgun, Mert Yuksekgonul, and James Zou. Cost-of-pass: An economic framework for evaluating language models. 2025.
- Reza Esfandiarpour, Vishwas Suryanarayanan, Stephen H. Bach, Vishal Chowdhary, and Anthony Aue. TheMCPCompany: Creating general-purpose agents with task-specific tools. 2025.
- Benjamin Estermann and Roger Wattenhofer. Reasoning effort and problem complexity: A scaling analysis in LLMs. 2025.
- Zikang Guo, Benfeng Xu, Chiwei Zhu, Wentao Hong, Xiaorui Wang, and Zhendong Mao. MCP-AgentBench: Evaluating real-world language agent performance with MCP-mediated tools. 2025.
- Marianne Menglin Liu, Daniel Garcia, Fjona Parllaku, Vikas Upadhyay, Syed Fahad Allam Shah, and Dan Roth. ToolScope: Enhancing LLM agent tool use through tool merging and context-aware filtering. 2025.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. 2023.
- Renze Lou, Baolin Peng, Wenlin Yao, Qianhui Wu, Hao Cheng, Suman Nath, Wenpeng Yin, and Jianfeng Gao. The tool illusion: Rethinking tool use in web agents. 2026. Published at COLM 2026.
- Elias Lumer, Vamse Kumar Subbiah, James A. Burke, Pradeep Honaganahalli Basavaraju, and Austin Huber. Toolshed: Scale tool-equipped agents with advanced RAG-tool fusion and tool knowledge bases. 2024.
- Mayug Maniparambil, Nils Hoehing, Janak Kapuriya, Arjun Karuvally, Ellen Rushe, Anthony Ventresque, Noel O’Connor, and Fergal Reid. TopoBench: Benchmarking LLMs on hard topological reasoning. 2026.
- Shinobu Miya. Eidoku: A neuro-symbolic verification gate for LLM reasoning via structural constraint satisfaction. 2025.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. 2023.
- Aninda Ray. Agent capsules: Quality-gated granularity control for multi-agent LLM pipelines. 2026.
- Hayley Ross, Ameya Sunil Mahabaleshwarkar, and Yoshi Suhara. When2Call: When (not) to call tools. 2025.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. 2023.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. 2023.

- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. 2023.
- Ningning Wang, Xavier Hu, Pai Liu, He Zhu, Yue Hou, Heyuan Huang, Shengyu Zhang, Jian Yang, Jiaheng Liu, Ge Zhang, Changwang Zhang, Jun Wang, Yuchen Eleanor Jiang, and Wangchunshu Zhou. Efficient agents: Building effective agents while reducing cost. 2025.
- Duo Wu, Jinghe Wang, Yuan Meng, Yanning Zhang, Le Sun, and Zhi Wang. CATP-LLM: Empowering large language models for cost-aware tool planning. 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. 2022.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. 2024.
- Takayuki Yato and Takahiro Seta. Complexity and completeness of finding another solution and its application to puzzles. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, E86-A(5):1052–1060, 2003.